

Day 11 — Random Forests & Model Comparison

Concepts · Code · Results · What It All Means
Target: Google · Meta · OpenAI · Anthropic

The Big Picture — What We Build Today

Yesterday you built a Logistic Regression model — one straight line separating hired from not hired. Today we build a Random Forest — hundreds of decision trees voting together. We also compare both models side by side to understand when each one wins. This is real ML engineering workflow.

File	What it does
day11_random_forest.py	Random Forest: build, train, evaluate, compare against Logistic Regression

Concept: What Is a Decision Tree?

A decision tree asks a series of yes/no questions to reach a prediction. Think of it like 20 questions. Each question splits the data into two groups. The tree learns which questions to ask and in what order from the training data.

Example tree for our hiring data:

```
experience > 10?
■■■ YES → education > 15?
■      ■■■ YES → HIRED (confidence: 92%)
■      ■■■ NO  → NOT HIRED (confidence: 78%)
■■■ NO  → income > 50000?
      ■■■ YES → education > 18?
      ■      ■■■ YES → HIRED
      ■      ■■■ NO  → NOT HIRED
      ■■■ NO  → NOT HIRED (confidence: 85%)
```

The tree learned experience matters most — it asked about experience first. This matches what we programmed: experience had weight 0.5, education had weight 0.3. Decision trees are naturally interpretable — you can trace exactly why a prediction was made.

Concept: What Is a Random Forest?

One decision tree is fragile — it memorises the training data and fails on new data. This is called overfitting. A Random Forest fixes this by building hundreds of trees and combining their votes.

How it works — 3 steps:

Step	What happens	Why it matters
1. Bootstrap sampling	Each tree trains on a random 80% sample of training data	Every tree sees slightly different data — creates diversity
2. Random feature selection	Each split considers only a random subset of features	Prevents all trees from learning the same patterns
3. Majority vote	500 trees each predict hired/not hired. Most votes win	Averaging cancels out individual tree errors

The wisdom of the crowd: 500 imperfect trees voting together beat one perfect-looking tree every time. This is why Random Forests almost always outperform single decision trees.

Concept: Key Parameters You Must Know

These are the knobs you turn when building a Random Forest. Understanding them is what separates someone who runs code from someone who understands ML.

Parameter	What it controls	Default	Rule of thumb
n_estimators	Number of trees	100	More trees = better, but slower. Start with 100-500.
max_depth	How deep each tree grows	None (unlimited)	Limit depth to prevent overfitting. Try 5-20.
min_samples_split	Min samples to split a node	2	Higher = simpler trees. Try 5-20 for noisy data.
max_features	Features considered per split	sqrt(n_features)	sqrt is good default for classification.
random_state	Seed for reproducibility	None	Always set this. Makes results reproducible.

Concept: Feature Importance in Random Forests

Random Forests give you feature importance for free — how much each feature contributed to the predictions across all 500 trees. More reliable than logistic regression coefficients because it captures non-linear relationships.

How importance is calculated:

Each time a feature is used to split a node, sklearn measures how much that split reduced prediction error. Sum this across all trees and all nodes for each feature. Normalise to sum to 1.0. Higher number = feature was more useful for making correct predictions.

Expected feature importance for our hiring data:

Feature	Expected importance	Why
experience	~0.50	Programmed weight 0.5 — model should find this
education	~0.30	Programmed weight 0.3 — model should find this
income	~0.15	Low signal — included as noise
age	~0.05	No signal — pure noise in our dataset

Concept: Model Comparison — When Does Each Win?

In ML you never just build one model. You always compare. The model that wins on your data is the one you ship. Here is how Logistic Regression and Random Forest differ:

	Logistic Regression	Random Forest
Accuracy	Good on linear data	Better on complex data
Speed	Very fast to train	Slower (500 trees)
Interpretability	Easy — one coefficient per feature	Harder — averaged over 500 trees
Overfitting	Low risk	Higher risk — control with max_depth
Feature scaling needed?	YES — must StandardScale	NO — trees are scale-invariant
Works on small data?	YES	Less reliable with very small datasets
Non-linear patterns?	NO — only straight lines	YES — trees capture curves and interactions

Important: Random Forest does NOT need StandardScaler. Decision trees split on thresholds — they don't care about the scale of features. This is a key difference from Logistic Regression.

New Code You Will See Today — Explained

1. RandomForestClassifier

Same sklearn API as yesterday: create, fit, predict. Only the model changes.

```
from sklearn.ensemble import RandomForestClassifier

# n_estimators=100: build 100 trees (more = better but slower)
# random_state=42: reproducibility – same result every run
# n_jobs=-1: use ALL your CPU cores to train in parallel
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

# Same fit/predict API as LogisticRegression
rf_model.fit(X_train, y_train)  # NO scaling needed
y_pred_rf = rf_model.predict(X_test)
```

2. Feature Importance

`rf_model.feature_importances_` is an array of floats summing to 1.0. One value per feature. Higher = more important.

```
# feature_importances_ is a NumPy array – one float per feature
# zip() pairs feature names with their importance scores
importances = rf_model.feature_importances_
for name, importance in zip(feature_names, importances):
    # :.3f means show 3 decimal places
    print(f'{name}: {importance:.3f}')
```

```
# argsort() returns indices that would sort the array
# [::-1] reverses it – so we get highest to lowest
sorted_idx = importances.argsort()[::-1]
```

3. zip() — pairing two lists together

zip() takes two lists and pairs them element by element. You will use this constantly in ML whenever you want to loop over names and values together.

```
names = ['age', 'income', 'education']
values = [0.03, 0.15, 0.81]

# Without zip – clunky index-based loop
for i in range(len(names)):
    print(names[i], values[i])

# With zip – clean and Pythonic
for name, value in zip(names, values):
    print(name, value)
```

4. argsort() — sorting by index

argsort() doesn't sort the values — it returns the indices that WOULD sort them. This lets you sort one array by the order of another.

```
import numpy as np

values = np.array([0.03, 0.15, 0.81, 0.50])
# argsort() returns: [0, 1, 3, 2]
# meaning: index 0 is smallest, then 1, then 3, then 2
idx = values.argsort()

# [::-1] reverses – now largest first: [2, 3, 1, 0]
idx_desc = values.argsort()[::-1]

names = ['age', 'income', 'education', 'experience']
# Sort names by importance (largest first)
sorted_names = [names[i] for i in idx_desc]
# Result: ['education', 'experience', 'income', 'age']
```

5. n_jobs=-1 — parallel processing

n_jobs=-1 tells sklearn to use all available CPU cores. Training 100 trees in parallel instead of one at a time. On your machine this could be 4-16x faster depending on CPU cores. You will see n_jobs=-1 in almost every sklearn model.

Full Code — Line by Line

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

[illegible]

[illegible]

```

axes[1].set_ylabel('Importance')

plt.tight_layout()
plt.savefig('day11_results.png', dpi=150, bbox_inches='tight')
plt.show()
print('\nChart saved as day11_results.png')

```

Expected Results

Metric	Logistic Regression	Random Forest
Accuracy	~86%	~88-92%
experience importance	coef ~2.30	~0.45-0.55
education importance	coef ~0.81	~0.25-0.35
Training time	Very fast	Slightly slower
Scaling required	YES	NO

Anki Cards — Day 11

Q: What is a decision tree?

A: A model that asks yes/no questions to reach a prediction

Q: What is a Random Forest?

A: Hundreds of decision trees voting together — majority wins

Q: What is bootstrap sampling?

A: Each tree trains on a random subset of training data — creates diversity

Q: Does Random Forest need StandardScaler?

A: NO — decision trees are scale-invariant

Q: What does `n_jobs=-1` do?

A: Uses all available CPU cores — trains trees in parallel

Q: What does `feature_importances_` return?

A: Array of floats summing to 1.0 — one per feature, higher = more important

Q: What does `argsort()[::-1]` do?

A: Returns indices sorted from largest to smallest value

Q: What does `zip()` do?

A: Pairs two lists element by element for parallel iteration

Q: When does Logistic Regression beat Random Forest?

A: Small datasets, linear data, when interpretability matters most

Q: What is overfitting?

A: Model memorises training data and fails on new data — too complex

Next: Day 12 — Gradient Boosting (XGBoost) and hyperparameter tuning